

ابزارهای عامل و تعامل پذیری

پروتکل‌هایی که عامل را از گاراژ بیرون می‌آورند

مطالعه پیشنهادی: ۵۰ دقیقه پیش‌نیاز: جزوه جلسه اول

فهرست مطالب

۱. چرا پروتکل؟ از ماشین سفارشی تا استاندارد صنعتی

۲. MCP از نگاه کدنویس شهودی

۳. مسئله $N \times M$ و حامل‌ها

۴. اشکال‌زدایی و جعبه‌ابزار مصرف امن

۵. تکامل معماری‌های عامل‌محور

۶. A2A: تعامل‌پذیری عامل به عامل

۷. A2UI: رابط کاربری به‌عنوان خروجی عامل

۸. تجارت عامل‌محور: UCP و AP2

۹. جمع‌بندی

۱۰. خودآزمایی

۱. چرا پروتکل؟ از ماشین سفارشی تا استاندارد صنعتی

تکامل بعدی نرم‌افزار نوشته نمی‌شود؛ به‌دست عامل‌های تعامل‌پذیر هماهنگ می‌شود.

در جلسه اول دیدیم که خروجی اصلی توسعه‌دهنده دیگر نحو خام نیست، بلکه سامانه‌ای است که کد تولید می‌کند. اگر مهندسی عامل‌محور همان کف کارخانه‌ای است که شما هماهنگش می‌کنید، MCP، A2A، A2UI و AP2 و UCP همان استانداردهای صنعتی آن‌اند: اندازه یکسان پیچ‌ومهره، قالب‌های داده و کانال‌های ارتباطی که اجازه می‌دهند ماشین‌آلات شما با بقیه جهان ایمن تعامل کنند.

هزینه نداشتن استاندارد

بدون پروتکل‌های باز، هر عاملی که می‌سازید یک «ماشین سفارشی» منزوی در گاراژ است. ناچارید برای هر ابزار و هر اتصال API پوشش‌های شکننده و اختصاصی بنویسید و نگهداری کنید. هر API می‌شود «استانداردی برای یک نفر» و این یعنی بدهی فنی خالص – کارهایی کم‌بازده که شما را در نقش رهبر ارکستر حبس می‌کنند.

پنج استاندارد و نقششان

استاندارد	استعاره	نقش
MCP پروتکل زمینه مدل	پریز USB-C درون مهار	اتصال فوری مدل‌ها به پایگاه‌های داده، فایل‌سیستم‌ها و API‌های وب.
مهارت‌ها	دفترچه راهنما	دستورها و اسکریپت‌های ساده‌ای که در محیط جعبه‌شن اجرا می‌شوند. موضوع جلسه سوم.
A2A عامل به عامل	بی‌سیم کارخانه	عامل‌های متخصص با آن مذاکره و هم‌فکری می‌کنند و کار را به هم می‌سپارند.
A2UI عامل به رابط کاربری	ویترین مولد	تبدیل خروجی خام و پیچیده به مؤلفه‌های بصری امن و تعاملی برای اپراتور انسانی.
UCP و AP2	شبکه تأمین و تراکنش جهانی	مذاکره و اجرای ایمن تراکنش‌های تجاری خودگردان.

تمرکز این جلسه: مصرف، نه ساخت

در مقاله پیشین این مجموعه، معماری سازمانی MCP – توپولوژی میزبان-کلاينت-سرور، ساخت سرور سفارشی و حاکمیت امنیتی – تشریح شده بود. اما برای کدنویس شهودی، اولویت مصرف است بر ساخت. شما نمی‌خواهید ساعت‌ها پیکربندی سرور بنویسید؛ می‌خواهید به فهرست‌های عمومی و خصوصی موجود وصل شوید و فوری به عامل ابرقدرت «وصل‌کن و کار کن» بدهید.

۲. MCP از نگاه کدنویس شهودی

روش سنتی سازمانی برای رساندن ابزار به مدل، سیم‌کشی دستی و اختصاصی است: هر اتصال تازه یعنی پوشش سفارشی REST، کلید API، سیاست تازه‌سازی توکن و تجزیه‌گر سفارشی JSON. MCP همه این اصطکاک را با یک سوکت استاندارد جایگزین می‌کند. راه‌اندازی در سه گام ساده انجام می‌شود.

گام ۱ – کشف

کدنویس شهودی اتصال‌دهنده سفارشی نمی‌نویسد؛ سرورهای از پیش ساخته‌شده را از سه منبع اصلی پیدا می‌کند:

- **فهرست‌های عمومی MCP:** صدها سرور آماده میزبانی می‌کنند. برای نمونه‌سازی سریع محلی عالی‌اند، اما بازبینی‌نشده‌اند و استفاده‌شان با مسئولیت خودتان است.
- **سرورهای راه دور شخص ثالث معتبر:** سکوه‌های بازبینی‌شده که نقاط پایانی میزبانی‌شده عرضه می‌کنند؛ مثلاً سرورهای رسمی منتشرشده برای سرویس‌های مدیریت‌شده که به عامل اجازه می‌دهند امن با آن سرویس‌ها تعامل کند.
- **فهرست‌های داخلی:** درون سازمان، ابزارهای داخلی به شکل سرور MCP عرضه و در فهرستی امن ثبت می‌شوند. پیاده‌سازی فنی این فهرست معمولاً از راه یک دروازه API، فهرست ثبت عامل، یا درگاه ریزسرویس خصوصی مدیریت می‌شود.

نکته کاربردی امنیتی

هنگام تصمیم‌گیری، همیشه امنیت را اولویت نخست بگذارید و تا جای ممکن سراغ دستورالعمل‌های رسمی بروید. اگر از سرورهای عمومی جامعه استفاده می‌کنید، **اعتبارنامه ندهید** و از سرویس‌های محافظ برای پیشگیری از مسائل امنیتی استفاده کنید.

گام ۲ – پیکربندی

پیش از اتصال، پارامترهای استاندارد دامنه و مجوزهای سرور را مشخص کنید:

- بررسی پیش‌نیازها.
- تعیین دامنه و معیارهای دسترسی.
- گنجاندن مشخصات در عامل کدنویسی.
- احراز هویت: تنظیم فایل‌های محیطی برای مدیریت اعتبارنامه‌ها (مانند توکن‌های دسترسی شخصی یا رازهای OAuth) و تعریف مجوزهای خواندن و نوشتن برای محافظت از فایل سیستم.

گام ۳ – اتصال

پس از پیکربندی، کلاینت اتصال را با سرور برقرار می‌کند. برای راستی‌آزمایی، کلاینت میزبان یک درخواست دست‌دادن ساده اجرا می‌کند تا ابزارهای موجود را فهرست کند و طرح‌واره خروجی را اعتبارسنجی کند.

۳. مسئله $N \times M$ و حامل‌ها

برای فهم اینکه چرا MCP برای نمونه‌سازی سریع حیاتی است، به ریاضیات یکپارچه‌سازی نگاه کنید. اگر با N مدل زبانی مختلف آزمایش می‌کنید و می‌خواهید آن‌ها را به M ابزار بیرونی وصل کنید، توسعه سنتی برای هر تقاطع مدل و ابزار کد یکپارچه‌سازی سفارشی می‌خواهد.

حساب ساده

با ۵ مدل و ۱۰ ابزار باید ۵۰ نقطه یکپارچه‌سازی اختصاصی را نگه دارید. اگر API یک ابزار عوض شود، چند حلقه تجزیه هم‌زمان می‌شکنند.

با پروتکل: پیچیدگی از $O(N \times M)$ به $O(N + M)$ کاهش می‌یابد.

حامل‌های استاندارد

با استاندارد شدن تعریف ابزارها، حامل‌ها را بدون نوشتن کد یکپارچه‌سازی مستقیماً به مهار عامل وصل می‌کنید:

حامل	سازوکار	مزایا و ملاحظات
stdio ورودی/خروجی استاندارد	کلاینت میزبان، سرور MCP را به‌عنوان زیرفرایند محلی در پس‌زمینه اجرا می‌کند و پیام‌های JSON-RPC 2.0 را از راه stdin و stdout رد و بدل می‌کند.	بیشتر برای کار محلی و نمونه‌سازی. عامل بدون پیکربندی پیچیده شبکه کار می‌کند و ابزار را مثل یک فرایند محلی می‌بیند.
SSE روی HTTP رویدادهای ارسالی از سرور	کلاینت میزبان (محلی یا یک برنامه عامل‌محور مستقر) به نقطه پایانی راه دور وصل می‌شود و داده به‌صورت زمان‌واقعی به عامل جاری می‌شود.	وابستگی کمتر، همیشه به‌روز، ردپای کوچک‌تر و چرخه عمر ساده‌تر؛ اما بار بیشتری روی سرور ابری می‌گذارد.

۴. اشکال زدایی و جعبه ابزار مصرف امن

وقتی عامل توهم می زند، لوله ها را ببینید

وقتی عامل پارامتر توهم می زند، ابزار اشتباه را صدا می کند یا نمی تواند بار داده را تجزیه کند، **وقتتان را صرف دست کاری کورکورانه دستورهای سامانه نکنید**. مستقیم لوله های انتقال را اشکال زدایی کنید:

- **MCP Inspector**: ابزار بومی توسعه دهنده که یک پنل وب محلی اجرا می کند. با آن می توانید هر سرور محلی یا راه دور را دستی پرس و جو کنید، طرحواره های ابزار فعال را ببینید، ورودی ها را دستی بیازمایید و بسته های خام JSON-RPC 2.0 را بازرسی کنید – بی آنکه گردش کار اصلی عامل را راه بیندازید.
- **Chrome DevTools**: مناسب محیط های توسعه وب محور و اشکال زدایی اتصال های SSE؛ امکان ردیابی جریان های ورودی و بررسی تأخیر سرور.

بایدها

- **سرورهای عمومی را پیش از اتصال ممیزی کنید**. پیش از وصل کردن سرور متن باز عمومی به عاملی که به فایل سیستم یا اعتبارنامه های شما دسترسی دارد، کدش را بخوانید.
- **برای ابزارها از بازیابی پویا استفاده کنید**. پنجره زمینه را تمیز نگه دارید: ابزار را فقط هنگام نیاز از فهرست بار کنید و پس از پایان کار از زمینه بیرون بیندازید تا توجه رقیق نشود.
- **از دروازه ها و فهرست های داخلی بهره بگیرید**. این یعنی مصرف طرحواره های تأیید شده و حکمرانی شده، به جای اختراع دوباره چرخ یا اجرای کد بازیابی نشده.
- **از بازرسی استفاده کنید**. وقتی عامل فراخوان ابزاری را با آرگومان های ساختگی می زند، به جای دست کاری کورکورانه درخواست سامانه، داده خام انتقال را ببینید.
- **انسان را در حلقه نگه دارید**. ورودی ابزار را پیش از فراخوانی سرور به کاربر نشان دهید تا از خروج تصادفی یا مخرب داده جلوگیری شود.
- **برای ممیزی ثبت کنید**. استفاده از ابزارها را گزارش برداری کنید.

نبایدها

- **اگر می شود مصرف کرد، نسازید**. در برابر وسوسه نوشتن پوشش سفارشی REST مقاومت کنید؛ اول دنبال سرور موجود بگردید.
- **سرور عمومی و تأیید نشده را در تولید به کار نبرید**. برای نمونه آخر هفته عالی اند، اما ریسک امنیتی و پایداری دارند. منطق اصلی کسب و کار را به نقطه پایانی تأیید نشده گره نزنید.

- **اعتبارنامه را سخت‌کد نکنید.** هرگز کلید یا توکن را مستقیم در درخواست عامل یا اسکریپت محلی نچسبانید؛ از متغیرهای محیطی استفاده کنید.
- **به محیط تولید وصل نشوید.** سرور را با پروژه توسعه به کار بگیرید، نه تولید. مدل‌ها در طراحی و آزمون عالی‌اند؛ از آن‌ها در محیطی امن و با داده غیرتولیدی یا مبهم‌سازی‌شده استفاده کنید.
- **برای به‌روزرسانی به کارش نبرید.** اگر ناچارید به داده واقعی وصل شوید، سرور را در حالت فقط‌خواندنی بگذارید.
- **دسترسی گسترده به همه پروژه‌ها ندهید.** دامنه سرور را به یک پروژه مشخص و منابع همان پروژه محدود کنید.

۵. تکامل معماری‌های عامل‌محور

برای فهم جایگاه کدنویسی شهودی در تکامل معماری‌های عامل‌محور، می‌توان الگویی تکرارشونده در فناوری را دید: گذار از پیکربندی دستی و سطح‌پایین، به هماهنگی اعلانی و مبتنی بر قصد – جایی که کاربر نمی‌گوید **چگونه** چیزی را بساز، بلکه می‌گوید **چه** می‌خواهد. همین روند در زیرساخت فناوری اطلاعات (گذار از پیکربندی دستی به «زیرساخت به‌مثابه‌کد») و در یادگیری ماشین (وعده AutoML برای خودکارکردن کار پرزحمت ساخت مدل) رخ داده است.

سقف معماری یکپارچه

امروز می‌توان کل یک برنامه را با کدنویسی شهودی ساخت. اما مسیر معماری‌های عامل‌محوری که این برنامه‌ها را می‌سازند، دارد یکی از مهم‌ترین گذارهای تاریخ نرم‌افزار را بازتاب می‌دهد: گذار از معماری یکپارچه به ریزسرویس‌ها. همان‌طور که برنامه‌های وب اولیه کدپایه‌هایی یگانه و غول‌آسا بودند که در آن‌ها همه کارکردها به هم چسبیده بود، بسیاری از پروژه‌های اولیه کدنویسی شهودی هم بر «عامل یکپارچه چاقوی همه‌کاره» تکیه می‌کنند: یک درخواست بسیار پیچیده که در آن یک عامل با ده‌ها ابزار متصل، همه‌چیز را از پرس‌وجوی پایگاه داده تا رندر رابط و آزمون بر عهده می‌گیرد. این معماری اجازه می‌دهد در یک آخر هفته نمونه‌ای سرهم شود، اما سرانجام به همان «سقف یکپارچگی» می‌خورد:

محدودیت	چه اتفاقی می افتد
اصطکاک مقیاس	نمی‌توانید «منطق پایگاه داده» را بهینه کنید بدون آنکه «منطق رابط کاربری» را که در همان درخواست است گیج کنید. افزون بر آن، هرچه ابزارهای در دسترس یک عامل بیشتر باشد، تصمیم‌گیری‌اش بدتر می‌شود: فضای جست‌وجوی کنش بعدی بیش‌ازحد بزرگ می‌شود و به پارامترهای ساختگی یا فراخوانی ابزار اشتباه می‌انجامد.
اضافه بار زمینه‌ای	چپاندن دستورهای کلان سامانه، ده‌ها طرح‌واره پیچیده ابزار و تاریخچه جاری گفت‌وگو در یک درخواست، به سرعت حافظه کاری مدل را پر می‌کند.
نقطه شکست یکتا	یک اشکال در یک ابزار یا یک دستور می‌تواند کل عامل را به توهم یا سقوط بکشاند. داده نامربوط یا خراب که در زمینه ذخیره شده، منتقل می‌شود و استدلال‌های بعدی را هم می‌شکند.

درسی که پیش‌تر آموخته شده بود

وقتی مدل‌های اولیه AutoML ارزش کسب‌وکاری‌شان را ثابت کردند، تیم‌ها به‌ندرت آن جعبه‌سیاه‌های خودتولید را در قلب کسب‌وکار رها کردند. برای رسیدن به مقیاس تولید، ناچار شدند جعبه را به مراحل قابل‌مشاهده و قابل‌نگه‌داری بشکنند – نسخه‌بندی داده، انبار ویژگی، تشخیص رانش. با تخصیص کردن اجزا، قانون بنیادی طراحی سامانه را به کار بستند: **تخصص، سازوکار مقیاس است.**

تخصص‌گرایی داخلی

تخصص‌گرایی داخلی یعنی افراز منطقی یک عامل یکپارچه به زیرعامل‌های متمایز و هدفمند. هر عامل با یک دستور سامانه بسیار متمرکز و زیرمجموعه‌ای مرتبط از ابزارها اداره می‌شود. این ساختار همچنان یکپارچه است، چون این زیرعامل‌های داخلی از مرز شبکه عبور نمی‌کنند؛ همان زمان اجرا و همان حافظه زیرین را به اشتراک می‌گذارند. نتیجه: پیمانه‌ای‌شدن تعریف‌های پیچیده، در کنار حفظ تأخیر پایین و مدیریت ساده وضعیت یک برنامه تک‌فرایندی. سه بهبود فنی مشخص:

- **کوچک‌شدن فضای جست‌وجو:** با محدود کردن ابزارها و مهارت‌های هر زیرعامل (مثلاً دادن فقط ابزارهای پرس‌وجو به عامل پایگاه داده)، خطاهای فراخوان ابزار و توهم به‌شدت کم می‌شود.
- **مهار رقیق‌شدن توجه:** تخصص به مدل زیرین اجازه می‌دهد روی یک دامنه واحد تمرکز کند و استدلال تیزتری داشته باشد.
- **بهینه‌شدن بار زمینه‌ای:** چون هماهنگ‌کننده به‌جای پردازش کل درخت منطق فقط کارها را مسیریابی می‌کند، زیرعامل‌ها نسبت سیگنال به نویز بالایی در پنجره زمینه‌شان حفظ می‌کنند.

معماری توزیع‌شده چندعاملی و پرسش «بسازیم یا بخریم»

با بلوغ معماری‌های عامل‌محور، جابه‌جایی چشمگیری به سوی معماری‌های توزیع‌شده در جریان است. رهبران صنعت از عرضه صرف API فراتر رفته‌اند و عامل‌های هوش مصنوعی دامنه‌ای منتشر می‌کنند که اکوسیستم اختصاصی خودشان را با دقت بومی می‌پیمایند.

از نظر فنی ممکن است سامانه‌ای چندعاملی با زیرعامل‌های سفارشی برای سکوهاى شخص ثالث بسازید، اما این کار **مالیات نگهداری** سنگینی می‌آورد: با انتخاب توسعه اختصاصی متخصص‌های دامنه، مسئولیت کامل به‌روزرسانی منطق درخواست و تعریف ابزارها در برابر تغییرات محصول و طرح‌واره API بالادست را خودتان بر عهده می‌گیرید. راهبرد معماری بهتر، بهره‌گیری از **عامل‌های متخصص رسمی** است که تیم‌های دارای تخصص عمیق دامنه نگاهشان می‌دارند.

اما گلوگاه تازه: پراکندگی

هر یک از این متخصص‌ها را ممکن است تیمی متفاوت با فناوری متفاوت ساخته باشد: یکی با یک چارچوب، دیگری با چارچوبی دیگر، سومی با چیزی کاملاً سفارشی. بیرون از مرزهای شبکه شما هستند، زبان‌های متفاوتی حرف می‌زنند، ساختار بار متفاوتی انتظار دارند، وضعیت گفت‌وگو را متفاوت پردازش می‌کنند و به لایه‌های انتقال گوناگون تکیه دارند. اگر برای هر متخصص ناچار باشید کد یکپارچه‌سازی سفارشی و حلقه تصحیح خطای اختصاصی بنویسید، مفهوم «تیم مجازی» فوری به یک پروژه یکپارچه‌سازی بدل می‌شود.

۶. A2A: تعامل‌پذیری عامل به عامل

این آشفتگی پراکندگی، دقیقاً همان چیزی است که پروتکل A2A برای استانداردکردنش ساخته شده. این پروتکل که ابتدا در گوگل توسعه یافت و سپس به بنیاد لینوکس اهدا شد، لایه‌ای جهانی از ارتباط برای سامانه‌های عامل‌محور معرفی می‌کند. A2A زبان مشترک اکوسیستم هوش مصنوعی است: ظرفیت‌های حامل شبکه، چارچوب‌های زیرین، زبان‌های برنامه‌نویسی و تفاوت‌های ساختار بار را انتزاع می‌کند. **همان‌طور که HTTP وب را استاندارد کرد، A2A نیروی کار مجازی را استاندارد می‌کند.**

چرا متخصص را نمی‌توان مثل ابزار صدا زد؟

«آیا فراخوان‌کننده به یک نتیجه نیاز دارد، یا به شرکت‌کننده دیگری که مسئولیت را بر عهده بگیرد؟»

صاحب‌خانه‌ای را تصور کنید که آشپزخانه‌اش را بازسازی می‌کند. دو انتخاب دارد: ابزار و دفترچه راهنما بخرد و خودش بسازد، یا پروژه را به متخصصی بسپارد که کارش ساختن آشپزخانه است. **ابزارها سازهای منفعل‌اند؛ متخصص**

شریکی همکار است. وقتی متخصص را استخدام می‌کنید، فقط یک نقشه دستش نمی‌دهید و نمی‌روید؛ او به حالت‌های مرزی یا اشتباهی در طرح اولیه شما برمی‌خورد که او را وامی‌دارد مکث کند، درباره موازنه‌ها با شما مشورت کند و بعد ادامه دهد.

دامنه کران‌دار در برابر بی‌کران

ابزار یا API استاندارد بر سازوکار «بفرست و فراموش کن» کار می‌کند: یک درخواست کاملاً قالب‌بندی‌شده انتظار دارد و یک پاسخ برمی‌گرداند. اما عامل در فضای حل مسئله بی‌کران کار می‌کند. محیط‌های نرم‌افزاری واقعی پر از ساختارهای داده مبهم، نیازمندی‌های گمراه‌کننده و ترجیح‌های متناقض‌اند – معادل دیجیتال دیوارهای کج. به ندرت می‌توان همه جزئیات لازم یک گردش کار پیچیده را بدون شفاف‌سازی چندنوبتی مشخص کرد.

مسئله GOTO در معماری عامل‌محور

چون دامنه عامل بی‌کران است، چپاندن آن در پوشش یک ابزار استاندارد، معادل معماری دستور GOTO را وارد هماهنگ‌کننده شما می‌کند. وقتی عاملی همکار را صدا می‌زنید، جریان کنترل از زمینه ساختارمند و مورد انتظار خارج می‌شود: عامل ممکن است به حالت وقفه برود، اطلاعات بیشتری بخواهد، و اگر کاربر نظرش عوض شود یا رهاش کند، شاید هرگز خروجی مورد انتظار را به فراخوان‌کننده برنگرداند.

راه‌حل: با ایزوله کردن این مسیریابی همکارانه در لایه A2A، لایه ابزار (MCP) تمیز، قابل‌پیش‌بینی و کاملاً ساختارمند باقی می‌ماند.

کارت عامل و فهرست‌های ثبت

برای آنکه کشف ممکن شود، هر عامل با یک **کارت عامل** تعریف می‌شود: «رزومه» استانداردشده دنیای هوش مصنوعی که هویتی ماشین‌خوان فراهم می‌کند و سه چیز را مشخص می‌کند: **توانایی‌ها** (چه کارهایی می‌تواند انجام دهد)، **امنیت و انطباق** (سیاست‌های مدیریت داده و مجوزهای لازم) و **طرح‌واره‌های تعامل** (دیگران چگونه باید با آن ارتباط بگیرند).

پس از تجهیز به کارت، عامل می‌تواند در یک **فهرست ثبت عامل** ثبت شود که هدفش تبدیل عامل‌ها به سرویس‌های کشف‌شدنی است:

- فهرست‌های عمومی (بازارگاه‌ها):** مانند یک آژانس استعدادیابی جهانی، به توسعه‌دهندگان اجازه می‌دهند عامل‌های متخصصشان را برای کشف‌شدن به جهان عرضه کنند. این راه را باز می‌کند که کسی برای یک صنعت خاص عامل متخصص بسازد و تخصصش را به هزاران هماهنگ‌کننده دیگر واگذار کند.
- فهرست‌های خصوصی:** درون سازمان، محیطی امن و حکمرانی‌شده فراهم می‌کنند. توسعه‌دهنده‌ای در شرکتی بزرگ می‌تواند عاملی بسازد که گردش‌کاری داخلی را خودکار می‌کند و آن را برای استفاده بخش‌های دیگر ثبت

کند، بی‌آنکه امنیت به خطر بیفتد.

پیاده‌سازی: دو حرکت توسعه

سمت عرضه – عرضهٔ عامل به‌عنوان عامل A2A:

- **تعریف کارت عامل:** مشخصات رسمی عامل.
- **پیاده‌سازی مجری عامل (لایهٔ ترجمه):** درخواست‌ها و پاسخ‌های A2A را به فراخوان‌های مشخص چارچوب عامل‌محور زیرین ترجمه می‌کند.
- **برپاکردن نقطهٔ پایانی A2A:** مجری باید به‌صورت نقطهٔ پایانی سازگار با پروتکل منتشر شود.

سمت تقاضا – اتصال به عامل‌های راه دور: برنامه به‌عنوان هماهنگ‌کننده عمل می‌کند؛ هسته‌ای مرکزی که بار شناختی‌اش صرف فهم قصد کاربر، مدیریت گردش کار کلان و واگذاری کارهای مشخص به عامل‌های تخصصی راه دور می‌شود. اتصال معمولاً یکی از دو الگو را دنبال می‌کند:

```
# 1. Direct instantiation via a hardcoded endpoint
# Ideal for specific vendor integrations or private agents
billing_specialist = RemoteA2aAgent(
    name="billing_agent",
    endpoint="https://api.vendor.com/v1/billing/a2a",
)

# 2. Indirect instantiation via an Agent Registry
# The registry resolves resource names and handles authentication validation
registry = AgentRegistry(project_id=project_id, location=location)
agent_name = f"projects/{project_id}/locations/{location}/agents/YOUR_AGENT_ID"
my_remote_agent = registry.get_remote_a2a_agent(agent_name=agent_name)
```

لایهٔ توسعه‌پذیری و درآمدزایی

با حل پراکندگی، A2A لایهٔ ارتباطی یکپارچه‌ای می‌سازد که می‌توان روی آن ساخت. هستهٔ پروتکل ستون فقرات انتقال و مذاکره است، اما برنامه‌های تراکنشی غنی اغلب توانایی‌های بسیار تخصصی می‌خواهند. این نیاز از راه سازوکار **افزونه‌های A2A** پاسخ می‌گیرد: الگویی استاندارد که به عامل‌ها اجازه می‌دهد قابلیت‌های اختیاری و مرتبه‌بالتر را امن اعلام، مذاکره و اجرا کنند. سه چارچوب بنیادی که به‌شکل افزونهٔ بومی روی این پایه کار می‌کنند عبارت‌اند از A2UI، UCP و AP2.

کارکردن عامل تنها نیمی از نبرد است؛ نیمه دیگر پایداری تجاری بلندمدت است. به دنبال موفقیت الگوی «نرم افزار به مثابه سرویس»، A2A به طور طبیعی مدل «عامل به مثابه سرویس» را ممکن می کند: عرضه عامل های تخصصی در مدل مصرف محور از راه کانال های فروش گوناگون، از بازارگاه های ابری با پشتیبانی بومی از قیمت گذاری ترکیبی (کارمزد ثابت به علاوه مصرف) تا ریزتراکنش های بدون مجوز ماشین به ماشین. در الگوی اخیر، سرور درخواست پرداخت نشده را با کد وضعیت HTTP 402 و صورت حسابی ماشین خوان برمی گرداند؛ عامل فراخوان، پرداخت را خودگردان انجام می دهد و درخواست را با توکن اثبات پرداخت رمزنگاشتی تکرار می کند.

۷. A2UI: رابط کاربری به عنوان خروجی عامل

شکاف ارتباطی

وقتی از همکاران می پرسید «عملکرد فصل چهارم به تفکیک منطقه چطور بود؟»، او صفحه گسترده دستتان نمی دهد. نموداری میله ای روی تخته می کشد، مناطق برجسته را دور می گیرد و زمینه اضافه می کند. انسان ها بینش را این طور به اشتراک می گذارند: با تصویر و تعامل. اما عامل های ما JSON خام برمی گردانند و بعد خودتان نمودار را می سازید: وارد کردن کتابخانه، پیکربندی محورها، مدیریت وضعیت. این جابه جایی مدام میان «پرسیدن برای بینش» و «کار رابط کاربری»، جریان کدنویسی شهودی را می شکند.

رابط کاربری مولد

رابط کاربری مولد یعنی مدل های زبانی در زمان اجرا و بر پایه قصد و زمینه کاربر، رابط بسازند. به جای آنکه توسعه دهنده هر حالت ممکن رابط را سخت کد کند، مدل رابط مناسب را به اقتضا تولید می کند. وقتی می پرسید «فروش فصل چهارم را به تفکیک منطقه مقایسه کن»، سامانه فقط داده بازیابی نمی کند: چیدمانی تعاملی، کارتها، صافی ها و کنترل های متناسب با همان درخواست را می سازد.

چالش: انجام امن این کار

اجرای کد دلخواه رابط کاربری که یک مدل زبانی تولید کرده، ریسک های امنیتی جدی دارد: تزریق کد، حمله XSS و عوارض جانبی کنترل نشده.

راه حل: اعلان قصد، نه تولید کد

A2UI استاندارد مستقل از چارچوب برای اعلان قصد رابط کاربری است: راهی متن باز که عامل ها رابط ها را در قالبی قابل حمل و اعلانی توصیف کنند، به جای جاری کردن داده خام یا فرستادن کد دلخواه.

آهنگ ساز به نوازنده ها فایل ضبط شده نمی دهد؛ نُت می دهد. همان نُت روی پیانو، ارکستر یا سینتی سایزر نواخته می شود و هر ساز نشانه گذاری را با صدای خودش تفسیر می کند.

A2UI نُت موسیقی رابط کاربری است: عامل قصد را می نویسد (چه چیزی رندر شود و چطور ترکیب شود) و هر رندرگری آن را به صورت بومی روی دستگاهی که کاربر واقعاً دارد اجرا می کند. عامل لازم نیست بداند مقصد، وب است یا موبایل یا پوشیدنی؛ فقط کاتالوگ مؤلفه های موجود را می داند.

چرا این جداسازی، A2UI را امن می کند

عامل کد اجرایی تولید نمی کند (که کابوس امنیتی است) و پیکسل از پیش رندر شده هم نمی فرستد (که نمی تواند بازچینش شود یا تعاملی بماند). به جایش از **کاتالوگ مورد اعتماد** درخواست مؤلفه می کند و کلاینت با کتابخانه مؤلفه خودش رندر می کند. کاتالوگ تعیین می کند چه چیزی در دسترس است، عامل تصمیم می گیرد چطور چیده شود، و کلاینت ساختار نهایی را سرهم می کند.

کاتالوگ پایه و ساخت کاتالوگ خودتان

A2UI یک کاتالوگ پایه با ۱۸ مؤلفه آماده استفاده برای نمونه سازی و نمایش عرضه می کند:

دسته	مؤلفه ها
چیدمان	Row, Column, List
نمایش	Text, Image, Icon, Divider
ظرف	Card, Modal, Tabs
رسانه	Video, AudioPlayer
تعاملی	Button, TextField, CheckBox, Slider, DateTimeInput, ChoicePicker

این مجموعه در نسخه ۰.۸ v0 «استاندارد» نامیده می شد و در ۰.۹ v0 به «پایه» تغییر نام یافت؛ نشانه ای عمدی که رابط های تولیدی باید **کاتالوگ خودشان** را بیاورند و مؤلفه های موجودشان (دکمه های سامانه طراحی، نمودارها، نقشه ها) را به انواع A2UI نگاشت کنند. عامل تغییر نمی کند؛ فقط نگاشت کاتالوگ رندرگر عوض می شود.

نمونه یک پیام A2UI (قالب نسخه ۰.۹ v0):

```

{
  "version": "v0.9",
  "updateComponents": {
    "surfaceId": "main",
    "components": [
      {
        "id": "root", "component": "Column",
        "children": ["title", "summary", "export"]},
      {
        "id": "title", "component": "Text",
        "text": "Q4 Sales", "variant": "h1"},
      {
        "id": "summary", "component": "Text",
        "text": "Revenue grew 12% QoQ"},
      {
        "id": "export", "component": "Button", "child": "export-label",
        "action": {"event": {"name": "export_csv"}}},
      {
        "id": "export-label", "component": "Text", "text": "Export CSV"}
    ]
  }
}

```

مؤلفه‌ها فهرستی مسطح از مجاورت‌ها را می‌سازند که با شناسه به هم ارجاع می‌دهند. همین ساختار، هم تولید تدریجی را برای مدل زبانی آسان می‌کند و هم به‌روزرسانی را برای کلاینت، بدون رندر دوبارهٔ کل صفحه.

دو الگوی تولید A2UI

در عمل دو راه برای تولید وجود دارد و انتخاب، بیشتر به این بستگی دارد که **تصمیم چیدمان کجا زندگی می‌کند**:

- **پیش‌فرض – مدل مستقیماً A2UI تولید می‌کند**: مدل مالک چیدمان است و آن را با قصد کاربر تطبیق می‌دهد؛ همان عامل برای «این منطقه‌ها را مقایسه کن» و «روندها را نشان بده» دو رابط متفاوت می‌سازد.
- **تخصصی – ابزار ساختار ثابتی برمی‌گرداند**: یک فراخوان ابزار، بدون صرف توکن برای ساخت رابط، با خروجی کاملاً قابل‌پیش‌بینی. انتخاب درست وقتی است که چیدمان از روی ورودی‌ها قطعی است: داشبورد فروش هر منطقه یک شکل دارد و هر فرم رزرو همان میدان‌ها را. ابزار در عمل یک قالب سمت سرور است.

پرسش	خروجی مناسب
«میانگین چقدر است؟»	داده به صورت متن
«این منطقه‌ها را مقایسه کن»	رابط تولیدشده به دست مدل زبانی
«داشبوردم را نشان بده»	رابط ساخته شده به دست ابزار (قالب ثابت)
تماس برنامه به برنامه	داده JSON

بوم تعاملی

رابط‌های گفت‌وگویی سنتی خطی‌اند و هر پاسخ ایستاست. رویکرد بوم متفاوت است: فضای کاری ماندگاری می‌سازد که هم عامل و هم کاربر می‌توانند ویرایشش کنند. به جای پیمایش تاریخچه گفت‌وگو، سندی زنده دارید؛ عامل بخش‌ها را تغییر می‌دهد، شما دستی ویرایش می‌کنید و هر دو تغییر در زمان واقعی بازتاب می‌یابد. **رابط کاربری فقط رندر نمی‌شود؛ خودش رسانه ارتباطی است.**

بهترین روش‌ها

- **دست‌نویسی JSON را کنار بگذارید.** از کیت توسعه رسمی استفاده کنید: مدیر طرح‌واره دستور سامانه‌ای می‌سازد که کاتالوگ و مثال‌های کارشده در آن تعبیه شده، کاتالوگ اعتبارسنج خودش را دارد و همان کیت، تجزیه‌گر بلوک‌های خروجی عامل را هم فراهم می‌کند.
- **اعتبارسنجی کنید و دوباره تلاش کنید.** در تولید، فراخوان ساخت رابط را در ساختار مدیریت خطا پیچید و در صورت شکست اعتبارسنجی طرح‌واره، به پاسخ متنی عقب‌نشینی کنید. خروجی مدل احتمالاتی است و رندرگر هرگز نباید بار بدشکل ببیند.
- **خروجی دوگانه بدهید.** هم داده و هم رابط را برگردانید تا مصرف‌کننده انتخاب کند: کلاینت‌های برنامه‌ای میدان رابط را نادیده می‌گیرند و کلاینت‌های انسانی آن را رندر می‌کنند.

۸. تجارت عامل‌محور: UCP و AP2

بخش‌های پیشین بر تعامل عامل‌ها با ابزارها، عامل‌های دیگر و رابط‌های کاربری تمرکز داشتند و این تعامل‌ها عمدتاً عملیات «خواندن» بودند. با تکامل این سامانه‌ها، عامل‌ها باید به اجرای «کنش‌هایی» گذر کنند که پیامد مالی واقعی دارند.

سناریوی راهنما

ساعت دو بامداد است و شما و هم‌خانه‌هایتان گرسنه‌اید، اما آن‌قدر مشغول درس‌خواندن هستید که نمی‌توانید متوقف شوید. پس دستیار هوشمندتان را می‌فرستید غذا بگیرد. ببینیم UCP و AP2 چطور کار را بین خودشان تقسیم می‌کنند.

UCP – پروتکل جهانی تجارت

پیش‌تر، هوش مصنوعی باید مرورگر را باز می‌کرد، به وب‌سایت بدطراحی رستوران می‌رفت، می‌کوشید تیک «گواکاموله اضافه» را بزند و امیدوار بود سایت از کار نیفتد. UCP مثل یک مترجم جهانی عمل می‌کند: هر رستوران منو، ساعت کاری و گزینه‌های سفارشی‌سازی را به زبان ماشینی تمیز و استاندارد منتشر می‌کند.

عامل شما با UCP از سامانه رستوران می‌پرسد «هنوز بازید؟ بوریتوی گیاهی دارید؟» و برای ساختن سفارش می‌گوید «یک بوریتوی مرغ، بدون پیاز، با خامه ترش، به‌علاوه یک چیپس»؛ و رستوران با مالیات، هزینه ارسال و زمان تخمینی تحویل پاسخ می‌دهد. **پیش از UCP، برنامه می‌توانست کشف و اطلاعات‌گیری کند، اما تعامل با هر تأمین‌کننده باید جداگانه هماهنگ می‌شد.**

AP2 – پروتکل پرداخت عامل‌ها

حالا غذا در سبد است و عامل باید پرداخت کند. بدیهی است که شماره کارت واقعی‌تان را در درخواست تایپ نمی‌کنید و نمی‌گویید «هر کاری خواستی بکن». AP2 پروتکل باز و مشترکی است که زبان مشترکی برای تراکنش‌های امن و منطبق میان عامل‌ها و فروشندگان فراهم می‌کند و اجازه می‌دهد عامل فقط درون قواعدی که شما تعیین کرده‌اید خرج کند:

- **حفاظ (مجوزنامه):** پیش از فرستادن عامل، یک قاعده دیجیتال را تأیید می‌کنید: «تا سقف ۲۵ دلار در این فروشگاه.»
- **دست‌دادن:** هنگام تسویه، شماره کارت نشان داده نمی‌شود؛ یک «سفته» دیجیتال رمزنگاشته و امضاشده ارائه می‌شود که می‌گوید «انسان من این سفارش ۱۸/۵ دلاری را تأیید کرده است». بانک فروشنده فوری این امضای دیجیتال را راستی‌آزمایی می‌کند.
- **بدون هزینه پنهان:** اگر رستوران بکوشد به جای ۱۸/۵ دلار، پنجاه دلار بگیرد، پروتکل فوری جلویش را می‌گیرد، چون قاعده‌ای را که شما امضا کرده‌اید نقض می‌کند.

AP2	UCP	
کیف پولی که امن می‌پردازد، بی‌آنکه سرتان کلاه برود.	مغزی که تصمیم می‌گیرد چه بخرد، منو را می‌فهمد و غذا را در سبد می‌گذارد.	نقش
سامانه‌های پرداخت	هر تأمین‌کننده کسب‌وکار	یکپارچگی با
مجوزدهی و ممیزی‌پذیری، اصالت قصد، مهار خطا و توهم عامل، پاسخ‌گویی	یکپارچگی واحد، زبان مشترک، معماری توسعه‌پذیر، رویکرد امنیت‌محور	ارزش‌های کلیدی

۹. جمع‌بندی

با پذیرش استانداردهای بنیادینی مانند MCP، A2A، A2UI، AP2 و UCP، سازمان‌ها می‌توانند بدهی فنی کمرشکن یکپارچه‌سازی‌های اختصاصی را حذف کنند و تمام تمرکزشان را روی هماهنگ‌کردن منطق باارزش کسب‌وکار بگذارند. این تغییر پارادایم، توسعه‌دهنده را از مکانیکی که API‌های شکننده را سیم‌کشی می‌کند، به معمار واقعی یک نیروی کار خودگردان جهانی ارتقا می‌دهد. با بلوغ این لایه‌های ارتباطی استانداردشده، صرفه‌های مقیاس کاملاً تازه‌ای گشوده می‌شود و شیوه ساخت، مصرف و درآمدزایی نرم‌افزار سازمانی دگرگون می‌شود.

۱۰. خودآزمایی

- در تیم شما چند مدل و چند ابزار در کارند؟ عدد $N \times M$ را حساب کنید. چند نقطه یکپارچه‌سازی سفارشی همین حالا دارید؟
- آخرین سروری که به عاملتان وصل کردید از کدام یک از سه منبع کشف بود؟ آیا کدش را خواندید؟
- برای یک کار محلی و یک کار مستقرشده، کدام حامل را انتخاب می‌کنید و چرا؟
- یک فراخوان ابزار شکست‌خورده را به یاد بیاورید. اگر آن روز بازرش داشتید، اول کدام سه چیز را نگاه می‌کردید؟
- کدام یک از شش «نباید» در پروژه فعلی شما همین حالا نقض می‌شود؟
- سه نشانه بگویید که عامل یکپارچه شما به سقف معماری خورده است.
- در پروژه خود، عامل را به کدام سه زیرعامل تخصصی می‌شکنید؟ هر کدام چه ابزارهایی باید داشته باشند و چه ابزارهایی نباید؟
- برای کدامیک از این‌ها MCP مناسب است و برای کدام A2A: تبدیل ارز، بازبینی قرارداد حقوقی، رزرو سفر چندمرحله‌ای؟ معیار تصمیم شما چیست؟
- چرا A2UI امن‌تر از این است که عامل مستقیم کد رابط تولید کند؟ در یک جمله پاسخ دهید.
- اگر قرار بود برای عامل خودتان یک «مجوزنامه» بنویسید، چه قیدهایی در آن می‌گذاشتید؟

منبع: این جزوه خلاصه و بازنویسی آموزشی مقاله Agent Tools & Interoperability نوشته کانچانا پاتلولا، لوکاش اولینیچاک و پیر پائولو ایپولیتو است (مجموعه مقالات عامل‌ها، می ۲۰۲۶). ترجمه کامل فارسی در همین پوشه در دسترس است.

دوره مهندسی عامل‌محور و کدنویسی شهودی – جزوه جلسه دوم